

Unit 3.5 Fast Fourier Transform

Computing and Visualizing the 2D DFT, FFT Derivation, Complexity Analysis,
Convolution, Correlation, and Padding

Digital Image Processing
Kiran Bagale

December, 2025

- | | |
|-----------------------------------|------------------------------------|
| 1 Fourier Transform Fundamentals | 1 1D FFT Derivation (Cooley-Tukey) |
| 2 2D DFT: Definition & Properties | 2 FFT Implementation Strategy |
| 3 Computing the 2D DFT | 3 Time Complexity: FFT vs DFT |
| 4 Time Complexity Analysis | 4 Convolution & Correlation |
| 5 Visualization Techniques | 5 Padding Strategies |

Key Learning Outcomes:

- Understand the mathematical foundation of Fourier analysis
- Compute and visualize 2D frequency spectra
- Implement efficient FFT algorithms
- Apply convolution and correlation in frequency domain

Fourier Analysis Timeline:

- **1822:** Joseph Fourier develops theory
- **1965:** Cooley-Tukey publish Fast Fourier Transform algorithm
- **1980s-1990s:** FFT becomes fundamental in signal/image processing
- **Today:** Essential tool in all digital domains

Why FFT Matters:

- Enables frequency domain analysis
- Dramatically reduces computation time
- Powers modern image/signal processing
- Essential for compression, filtering, analysis

Core Insight

Any signal can be decomposed into sum of sinusoids at different frequencies

1D Discrete Fourier Transform (DFT)

Definition: For a discrete signal $x[n]$, $n = 0, 1, \dots, N - 1$

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N - 1$$

Inverse DFT (IDFT):

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{j2\pi kn/N}, \quad n = 0, 1, \dots, N - 1$$

- $X[k]$ is the complex frequency component at frequency bin k
- $e^{-j2\pi kn/N}$ are the basis functions (complex exponentials)
- $j = \sqrt{-1}$

Alternative Form: $X[k] = A[k]e^{j\phi[k]}$

- **Magnitude:** $|X[k]| = \sqrt{\text{Re}^2[X[k]] + \text{Im}^2[X[k]]}$
- **Phase:** $\phi[k] = \arctan(\text{Im}[X[k]]/\text{Re}[X[k]])$

2D Discrete Fourier Transform

Definition: For a 2D signal (image) $x[m, n]$, $m, n = 0, 1, \dots, N - 1$

$$X[k, l] = \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} x[m, n] \cdot e^{-j2\pi(km+ln)/N}$$

2D IDFT:

$$x[m, n] = \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} X[k, l] \cdot e^{j2\pi(km+ln)/N}$$

Properties:

- **Separability:** Can compute as 1D DFT per row, then per column
- **Linearity:**
 $\mathcal{F}\{a \cdot x + b \cdot y\} = a\mathcal{F}\{x\} + b\mathcal{F}\{y\}$
- **Symmetry:** For real $x[m, n]$:
 $X[N - k, N - l] = X^*[k, l]$
- **Parseval:** Energy conserved in both domains

Method 1: Direct Computation

Algorithm 1 Direct 2D DFT

Input: $x[m, n]$ ($N \times N$ image)

Output: $X[k, l]$ (DFT coefficients)

```
for  $k = 0$  to  $N - 1$  do
  for  $l = 0$  to  $N - 1$  do
     $X[k, l] \leftarrow 0$ 
    for  $m = 0$  to  $N - 1$  do
      for  $n = 0$  to  $N - 1$  do
         $\theta \leftarrow 2\pi(km + ln)/N$ 
         $X[k, l] \leftarrow X[k, l] + x[m, n] \cdot e^{-j\theta}$ 
      end for
    end for
  end for
end for
```

Time Complexity: $\mathcal{O}(N^4)$ operations

- Four nested loops: $N \times N \times N \times N$
- For $N = 1024$: $\approx 1.1 \times 10^{12}$ operations
- For $N = 1024$: ≈ 280 seconds at 4 GHz

2D DFT via Separability

Key Property: Separability

$$X[k, l] = \sum_{n=0}^{N-1} e^{-j2\pi ln/N} \left(\sum_{m=0}^{N-1} x[m, n] \cdot e^{-j2\pi km/N} \right)$$

Algorithm: Row-Column Decomposition

Step 1: Compute 1D DFT of each row

$$X_{\text{row}}[m, k] = \sum_{n=0}^{N-1} x[m, n] e^{-j2\pi kn/N}$$

Step 2: Compute 1D DFT of each column

$$X[k, l] = \sum_{m=0}^{N-1} X_{\text{row}}[m, k] e^{-j2\pi lm/N}$$

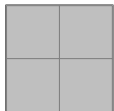
Time Complexity: $\mathcal{O}(N^3)$ operations

- Two 1D DFTs per row/column: N rows $\times$ N ops = N^2 operations
- Then N columns $\times$ N ops = N^2 operations
- Total: $2N^3$ for direct method
- For $N = 1024$: $\approx 2 \times 10^9$ operations (≈ 0.5 seconds)

Visualizing the 2D DFT

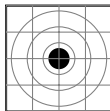
Magnitude Spectrum Representation:

Original Image



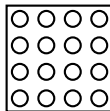
$$x[m, n]$$

Magnitude Spectrum



$$|X[k, l]| \text{ (log scale)}$$

Phase Spectrum



$$\phi[k, l]$$

DC Component (Zero Frequency):

- Located at $[0, 0]$ or center (after FFT shift)
- Represents average intensity/brightness
- Usually largest component in magnitude spectrum

Directional Information:

- Horizontal edges $\rightarrow$ vertical frequency components
- Vertical edges $\rightarrow$ horizontal frequency components
- Diagonal patterns $\rightarrow$ diagonal frequency bands

DFT Time Complexity Analysis

Complexity Comparison Table

Method	Complexity	N=256	N=512	N=1024
Direct 2D DFT	$\mathcal{O}(N^4)$	4.3×10^9	6.9×10^{10}	1.1×10^{12}
Row-Column (1D)	$\mathcal{O}(N^3)$	1.7×10^7	1.3×10^8	2.1×10^9
2D FFT	$\mathcal{O}(N^2 \log N)$	2.1×10^6	2.3×10^7	2.0×10^8

Speedup Factor (vs Direct DFT):

- Row-Column: $\mathcal{O}(N)$ faster than direct
- FFT: $\mathcal{O}(N^2 / \log N)$ faster than direct
- For $N = 1024$: FFT is $\approx 5500\times$ faster!

Computational Time Estimates (at 4 GHz):

	Direct	Row-Col	FFT
N=256	1.1 ms	0.004 ms	0.0005 ms
N=1024	280 sec	0.5 sec	50 ms

Why Do We Need FFT?

The DFT Problem:

- Direct computation requires N^2 multiplications
- For $N = 2^{10} = 1024$: 1 million multiplications
- For real images (2048×2048): $> 10^{13}$ operations
- Impractical for real-time applications

Key Insight for FFT:

Exploit Periodicity and Symmetry

$$e^{-j2\pi k(n+N/2)/N} = e^{-j2\pi kn/N} \cdot e^{-j\pi k}$$

Leads to $N \log N$ algorithm by recursively computing DFT of even/odd indices

Impact:

- Reduces complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$
- For $N = 1024$: from 1M to 10K operations (100× faster)
- Makes frequency domain processing practical

Cooley-Tukey FFT Derivation (1D)

Starting with DFT definition:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad \text{where } W_N = e^{-j2\pi/N}$$

Divide into even and odd indices:

$$X[k] = \sum_{r=0}^{N/2-1} x[2r] W_N^{k(2r)} + \sum_{r=0}^{N/2-1} x[2r+1] W_N^{k(2r+1)}$$

Simplify using $W_N^2 = W_{N/2}$:

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{kr}}_{\text{even part}} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{kr}}_{\text{odd part}}$$

Cooley-Tukey FFT Derivation (1D)

Denoting:

$E[k]$ = DFT of even-indexed samples

$O[k]$ = DFT of odd-indexed samples

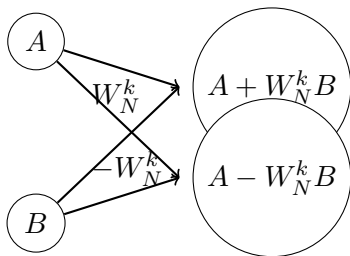
Butterfly Operation:

$$X[k] = E[k] + W_N^k \cdot O[k], \quad k = 0, 1, \dots, N/2 - 1$$

$$X[k + N/2] = E[k] - W_N^k \cdot O[k]$$

FFT Butterfly Operation

Butterfly Computation:



Complexity of One Butterfly:

- 1 complex multiplication: $W_N^k \cdot B$
- 2 complex additions: $(A \pm W_N^k B)$
- Total: 1 complex mult + 2 complex adds per butterfly

FFT Structure (for $N = 2^m$):

- Number of stages: $\log_2 N$ stages
- Butterflies per stage: $N/2$
- Total butterflies: $(N/2) \times \log_2 N$
- Total complexity: $\mathcal{O}(N \log N)$

FFT Algorithm (Cooley-Tukey, Radix-2)

Algorithm 2 Radix-2 FFT (Decimation-in-Time)

Input: $x[n]$, $N = 2^m$ **Output:** $X[k]$ (DFT)

Step 1: Bit-reversal permutation of $x[n]$

```
for  $k = 0$  to  $m - 1$  do
     $\ell \leftarrow 2^k$  (stage width)
    for  $j = 0$  to  $\ell - 1$  do
         $\theta \leftarrow 2\pi j / (2\ell)$ 
         $W \leftarrow e^{-j\theta}$ 
        for  $i = 0$  to  $N/(2\ell) - 1$  do
             $p \leftarrow i \cdot 2\ell + j$ ;  $q \leftarrow p + \ell$ 
             $u \leftarrow x[p]$ 
             $v \leftarrow x[q] \cdot W$ 
             $x[p] \leftarrow u + v$ 
             $x[q] \leftarrow u - v$ 
        end for
    end for
end for
Return:  $X = x$  (in-place computation)
```

FFT Example: $N = 4$

4-Point FFT Computation:

Step 1: Bit-Reversal Permutation

Original indices: $[0, 1, 2, 3] \rightarrow$ Bit-reversed: $[0, 2, 1, 3]$

Step 2: Stage 1 - 2-Point FFTs

- Pairs: $(x[0], x[2])$ and $(x[1], x[3])$ with simple add/subtract
- Outputs: $F_1[0], F_1[1], F_1[2], F_1[3]$

Step 3: Stage 2 - Combine with Twiddle Factors

- Apply twiddle factors: $W_4^k = e^{-j2\pi k/4}$
- Cross-combine results from Stage 1
- Final outputs: $X[0], X[1], X[2], X[3]$

Complexity Comparison:

Method	Multiplications	Speedup
Direct DFT (N^2)	16	$1\times$
Radix-2 FFT ($\frac{N}{2} \log_2 N$)	4	$4\times$

Key Observations:

- Number of stages: $\log_2(4) = 2$
- Butterflies per stage: 2
- Total butterflies: $2 \times 2 = 4$
- Speedup for $N = 4$: $4\times$ (generalizes to $O(N/\log N)$ speedup)

FFT Time Complexity Analysis

Detailed Complexity Breakdown:

	Direct DFT	Radix-2 FFT	Ratio	Factor
Multiplications	N^2	$\frac{N}{2} \log_2 N$	$\frac{N}{2 \log_2 N}$	$\mathcal{O}(N / \log N)$
Additions	$N(N - 1)$	$N \log_2 N$	$\frac{N}{\log_2 N}$	$\mathcal{O}(N / \log N)$

Numerical Examples:

N	DFT Mults	FFT Mults	Speedup	Time Saved (4GHz)
256	65,536	1,024	64×	15.3 s
512	262,144	2,304	114×	61 s
1024	1,048,576	5,120	205×	255 s
2^{20}	2^{40}	10^7	$\approx 10^8 \times$	seconds → nanoseconds

FFT Asymptotic Complexity

$$\mathcal{O}(N \log N) \quad \text{vs.} \quad \mathcal{O}(N^2) \text{ for DFT}$$

1D Convolution (Linear):

$$y[n] = (x * h)[n] = \sum_{m=0}^{M-1} h[m] \cdot x[n - m]$$

2D Convolution (Images):

$$g[m, n] = (f * h)[m, n] = \sum_{i=0}^{I-1} \sum_{j=0}^{J-1} h[i, j] \cdot f[m - i, n - j]$$

Frequency Domain Convolution

Convolution in time domain = Multiplication in frequency domain

$$g[m, n] = f[m, n] * h[m, n] \quad \Leftrightarrow \quad G[k, l] = F[k, l] \cdot H[k, l]$$

Why This Matters:

- Time domain convolution: $\mathcal{O}(MN \cdot IJ)$ complexity
- Frequency domain: 1 FFT + multiply + 1 IFFT = $\mathcal{O}(N \log N)$ per FFT
- Much faster for large kernels

Correlation in Frequency Domain

2D Correlation:

$$\rho[m, n] = (f \circ h)[m, n] = \sum_{i=0}^{I-1} \sum_{j=0}^{J-1} h[i, j] \cdot f[m + i, n + j]$$

Relationship to Convolution:

$$\rho[m, n] = (f * h^*)[m, -n] \quad (\text{flip and reverse})$$

Correlation Theorem:

Frequency Domain Correlation

$$\rho[m, n] = f[m, n] \circ h[m, n] \quad \Leftrightarrow \quad P[k, l] = F[k, l] \cdot H^*[k, l]$$

(where H^* is complex conjugate of H)

Applications:

- **Cross-correlation:** Measure similarity between two signals
- **Auto-correlation:** Measure self-similarity (useful for periodicity detection)
- **Template matching:** Find pattern location in image

Normalized Cross-Correlation:

$$r[m, n] = \frac{\rho[m, n] - \mu_f \mu_h}{\sigma_f \sigma_h}$$

(ranges from -1 to 1, independent of intensity scaling)

Circular vs. Linear Convolution

Linear Convolution:

$$y[n] = \sum_{m=0}^{M-1} h[m] \cdot x[n - m],$$

where $n = 0, \dots, N+M-2$

- Output length: $N + M - 1$ samples
- **No wraparound** at boundaries
- Used in most signal/image processing

Circular Convolution:

$$y_c[n] = \sum_{m=0}^{N-1} h[m] \cdot x[(n - m) \bmod N],$$

where $n = 0, \dots, N-1$

- Output length: N samples (same as input)
- **With wraparound** at boundaries
- Natural result of DFT/FFT multiplication

Issue: Circular Convolution Aliasing

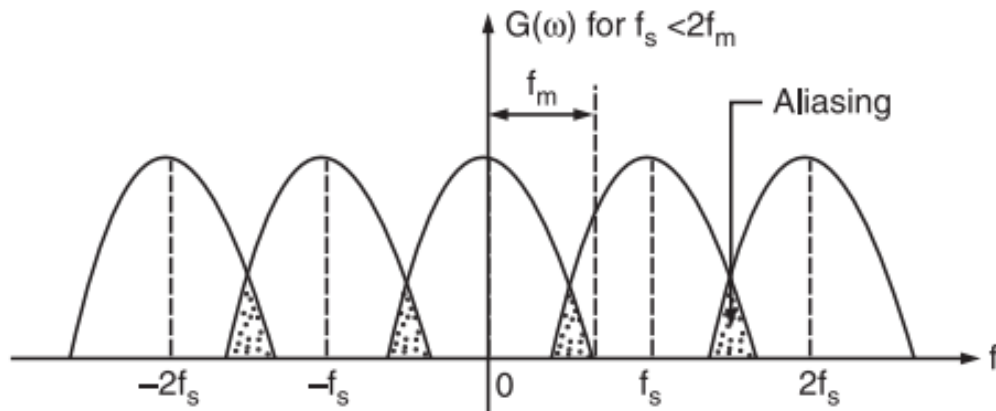


Figure: Spectrum of the sampled signal for the case $f_s < 2f_m$

Zero Padding

Problem: Circular Convolution Aliasing

- FFT assumes periodic/circular signals
- Circular convolution wraps around boundaries
- Non-overlapping regions incorrectly combined

Solution: Zero Padding

Extend signals with zeros to prevent aliasing:

$$\text{Pad size} = N_x + N_h - 1$$

- If $x[n]$ has length N_x and $h[m]$ has length N_h
- Pad both to length $P \geq N_x + N_h - 1$ with zeros
- Then circular convolution equals linear convolution

Example:

- $x[n]$: 100 samples
- $h[m]$: 25 samples
- Pad to: $100 + 25 - 1 = 124$ samples
- Use FFT size: 256 (next power of 2)

Algorithm:

- 1 Pad x and h to size 256
- 2 Compute $X = \text{FFT}(x)$
- 3 Compute $H = \text{FFT}(h)$
- 4 Multiply: $Y = X \cdot H$
- 5 Inverse: $y = \text{IFFT}(Y)$
- 6 Extract first 124 samples

Padding Strategies in Image Processing

Different Padding Methods:

Method	Boundary Extension	Use Case
Zero Padding	$f[i, j] = 0$ (outside bounds)	FFT-based convolution
Periodic Extension	$f[i \bmod M, j \bmod N]$	Circular convolution
Reflection	Mirror at boundaries	Reduce boundary artifacts
Replication	Copy edge values	Natural for photographs

Frequency Domain Filtering:

- **FFT filtering:** Zero padding prevents circular convolution errors
- **Spatial filtering:** Reflection padding preferred (less artifacts)
- **Trade-off:** Computational cost vs. visual quality

Algorithm 3 FFT-based Image Filtering

Input: Image $f[m, n]$ ($M \times N$), Filter kernel $h[i, j]$ ($P \times Q$)

Output: Filtered image $g[m, n]$

▷ Step 1: Determine FFT size

$F_m \leftarrow 2^{\lceil \log_2(M+P-1) \rceil}$ (next power of 2)

$F_n \leftarrow 2^{\lceil \log_2(N+Q-1) \rceil}$

▷ Step 2: Zero-pad inputs

$f_{\text{pad}}[m, n] \leftarrow f[m, n]$ with padding to $(F_m \times F_n)$

$h_{\text{pad}}[i, j] \leftarrow h[i, j]$ with padding to $(F_m \times F_n)$

▷ Step 3: Frequency domain operations

$F[k, l] \leftarrow \text{FFT}(f_{\text{pad}})$

$H[k, l] \leftarrow \text{FFT}(h_{\text{pad}})$

$G[k, l] \leftarrow F[k, l] \cdot H[k, l]$

▷ Step 4: Return to spatial domain

$g[m, n] \leftarrow \text{IFFT}(G[k, l])$

▷ Step 5: Extract valid region

Return: $g[0 : M - 1, 0 : N - 1]$

Computational Advantage:

- Spatial filtering: $\mathcal{O}(M \cdot N \cdot P \cdot Q)$
- FFT filtering: $\mathcal{O}(F_m \log F_m \cdot F_n \log F_n)$
- When $P \cdot Q$ is large, FFT is much faster

Complexity Summary:

Method	Complexity	Relative Speed
Direct 2D DFT	$\mathcal{O}(N^4)$	$1\times$
Row-Column	$\mathcal{O}(N^3)$	$N\times$
2D FFT	$\mathcal{O}(N^2 \log N)$	$N^2 / \log N \times$

Lecture 1 Highlights:

- ① **2D DFT** decomposes images into frequency components
- ② **Separability** reduces 2D DFT from $\mathcal{O}(N^4)$ to $\mathcal{O}(N^3)$
- ③ **Visualization** reveals directional features and patterns

Lecture 2 Highlights:

- ① **Cooley-Tukey FFT** algorithm reduces to $\mathcal{O}(N \log N)$
- ② **Butterfly operations** are the building blocks of FFT
- ③ **Convolution theorem** enables frequency domain filtering
- ④ **Zero padding** prevents circular convolution aliasing

Thank You!